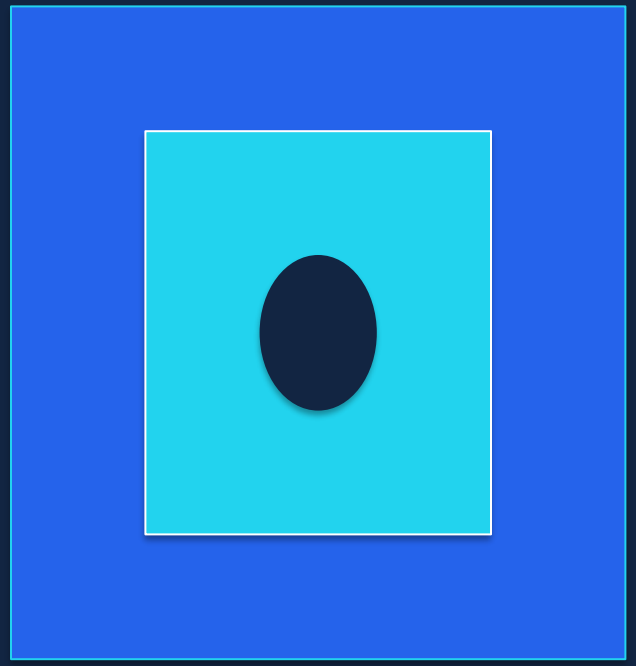
EdgeAI_Vision

Da API funcionando até a criação da IA

Desconstruindo o projeto de Visão Computacional de Borda



Nossa investigação começa pelo resultado

Primeiro vemos a solução funcionando; depois abrimos cada camada.

1 • O QUÊ?

Uma API recebe imagens e devolve detecções de EPIs.

2 • PRA QUÊ?

Separar responsabilidades facilita testar, corrigir e evoluir.

3 • COMO?

Voltaremos da API ao treinamento e aos scripts criados com IA.

O resultado final é uma cadeia de peças que trabalham juntas.



Etapa 1 — A API em funcionamento

O ponto de partida: uma aplicação acessível pela rede.

http://raspberry-pi:8000/

```
{ "service": "EdgeAI_Vision API",
    "status": "online",
    "mock_mode": false,
    "classes": ["pessoa", "capacete",
"colete", "luva", "oculos"] }
```

Exemplo de resposta esperada

O QUÊ?

Serviço que recebe pedidos HTTP.

PRA QUÊ?

Outro programa pede análise sem conhecer o modelo.

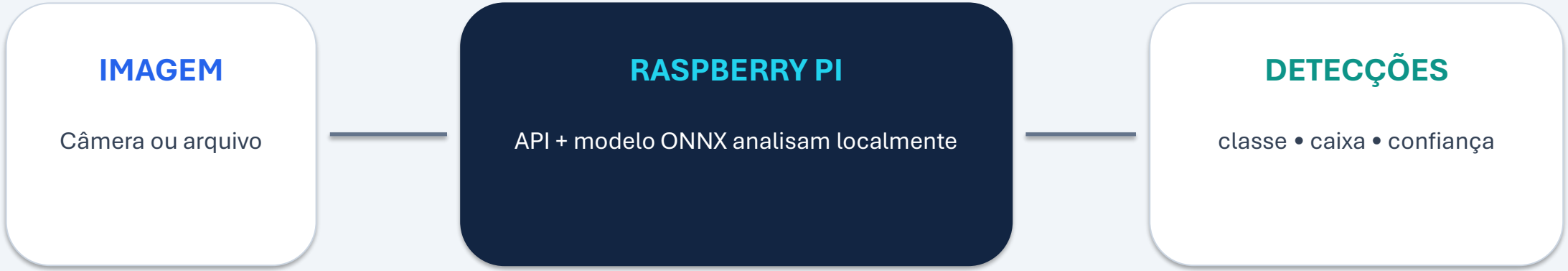
COMO?

Executada no Raspberry Pi, dentro de um container Docker.

mock_mode=false significa que a aplicação não deve responder com detecções simuladas.

Etapa 2 — Visão Computacional de Borda

A análise acontece perto de onde a imagem é capturada.



O QUÊ?

IA analisa imagens para localizar objetos.

PRA QUÊ?

Resposta rápida e menor dependência da nuvem.

COMO?

A imagem é enviada à API no dispositivo de borda.

Visão Computacional = técnicas para o computador interpretar imagens.

Etapa 3 — Dois endpoints, duas formas de ver

Endpoint = endereço da API que executa uma função específica.

POST /predict

O QUÊ? Devolve dados estruturados (JSON).

PRA QUÊ? Outro software usa classes, coordenadas e confiança.

COMO? Envia-se imagem numa requisição HTTP; a resposta é estruturada.

POST /predict/image

O QUÊ? Devolve imagem com detecções desenhadas.

PRA QUÊ? Facilitar inspeção visual e demonstração.

COMO? A API processa e retorna a imagem anotada.

JSON = dados

HTTP = comunicação

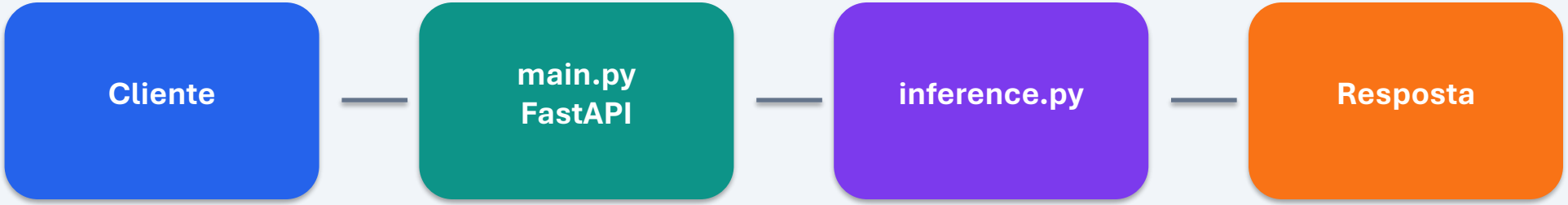
POST = enviar pedido

Imagem anotada

A interface da API pode permanecer igual mesmo quando trocamos o modelo.

Etapa 4 — main.py: o porteiro

O arquivo principal recebe pedidos e encaminha o trabalho.



O QUÊ?

Define rotas, valida pedidos e monta respostas.

PRA QUÊ?

Separar comunicação da lógica de visão computacional.

COMO?

Recebe a imagem e chama o componente de inferência.

FastAPI = ferramenta Python para construir APIs web.

Etapa 5 — inference.py: o especialista

Aqui a imagem é preparada, analisada e convertida em detecções.



O QUÊ?

Código que executa a inferência — previsão do modelo.

PRA QUÊ?

Transformar pixels em classes, caixas e confiança.

COMO?

Prepara entrada, roda modelo e filtra resultados.

Pré-processamento = preparar a entrada. Pós-processamento = interpretar e filtrar a saída.

Etapa 6 — Docker: a caixa organizada

O ambiente é empacotado para executar de forma mais reproduzível.

Dockerfile

Receita que define como montar o ambiente.

Imagem Docker

Pacote criado a partir da receita.

Container

Aplicação em execução com suas dependências.

docker-compose.yml

Configuração para iniciar e ajustar o serviço.

Por que usar?

Facilita reproduzir, reiniciar e testar o ambiente.

Dependência = biblioteca ou componente de que a aplicação precisa.

Etapa 7 — ONNX: o modelo pronto para rodar

O arquivo de inferência é exportado a partir do checkpoint treinado.

best.pt

pesos do melhor checkpoint

Exportação

conversão de formato

best.onnx

modelo para inferência

O QUÊ?

ONNX = formato aberto para representar modelos de IA.

PRA QUÊ?

Executar o modelo fora do ambiente de treinamento.

COMO?

Exporta-se checkpoint e configura-se inference.py.

Checkpoint = arquivo que guarda os parâmetros aprendidos durante o treinamento.

Etapa 8 — O dataset: exemplos para aprender

Dataset = conjunto de imagens e anotações que ensina o modelo.



PESSOA

corpo humano



CAPACETE

cabeça



COLETE

tronco



LUVA

mãos



ÓCULOS

olhos

O QUÊ?

Imagens + caixas que marcam onde está cada objeto.

PRA QUÊ?

Aprender padrões visuais das cinco classes.

COMO?

Dataset SH17 filtrado/organizado para as classes do projeto.

Qualidade, variedade e anotações corretas influenciam o que o modelo consegue aprender.

Etapa 9 — YOLO Small e o arquivo YAML

YOLO detecta objetos; YAML informa onde estão os dados e quais são as classes.

YOLO Small

O QUÊ? Variante de tamanho small da família YOLO.

PRA QUÊ? Buscar equilíbrio entre capacidade de detecção e custo computacional.

COMO? Foi usado como modelo-base no novo treinamento.

data.yaml

O QUÊ? Arquivo de configuração dos dados.

PRA QUÊ? Declarar caminhos e nomes de classes.

COMO? Define treino/validação e mapeamento das 5 classes.

YOLO = detector

YAML = configuração

Small ≠ Nano

Nano costuma ser mais leve; Small é maior e pode aprender padrões mais complexos, mas tende a exigir mais recursos. O resultado deve ser medido no dispositivo-alvo.

Etapa 10 — Treinar por 10 epochs no Colab

Epoch = uma passagem completa do conjunto de treino pelo modelo.

Google Colab

ambiente na nuvem

NVIDIA T4

GPU gratuita*

10 epochs

10 passagens

O QUÊ?

Ajustar parâmetros internos com exemplos anotados.

PRA QUÊ?

Reduzir erros e reconhecer objetos melhor.

COMO?

Pipeline executada no Colab com GPU NVIDIA Tesla T4.

*Disponibilidade gratuita varia. Dez epochs são um experimento inicial, não garantia de desempenho ideal.

Etapa 11 — O que os resultados dizem?

Valores do results2.csv no epoch 10; cada métrica responde a uma pergunta diferente.

Precision

78.8%

Das detecções feitas, quantas estavam corretas?

Recall

68.5%

Dos objetos reais, quantos foram encontrados?

mAP50

72.9%

Qualidade média com IoU de 0,50.

mAP50–95

50.9%

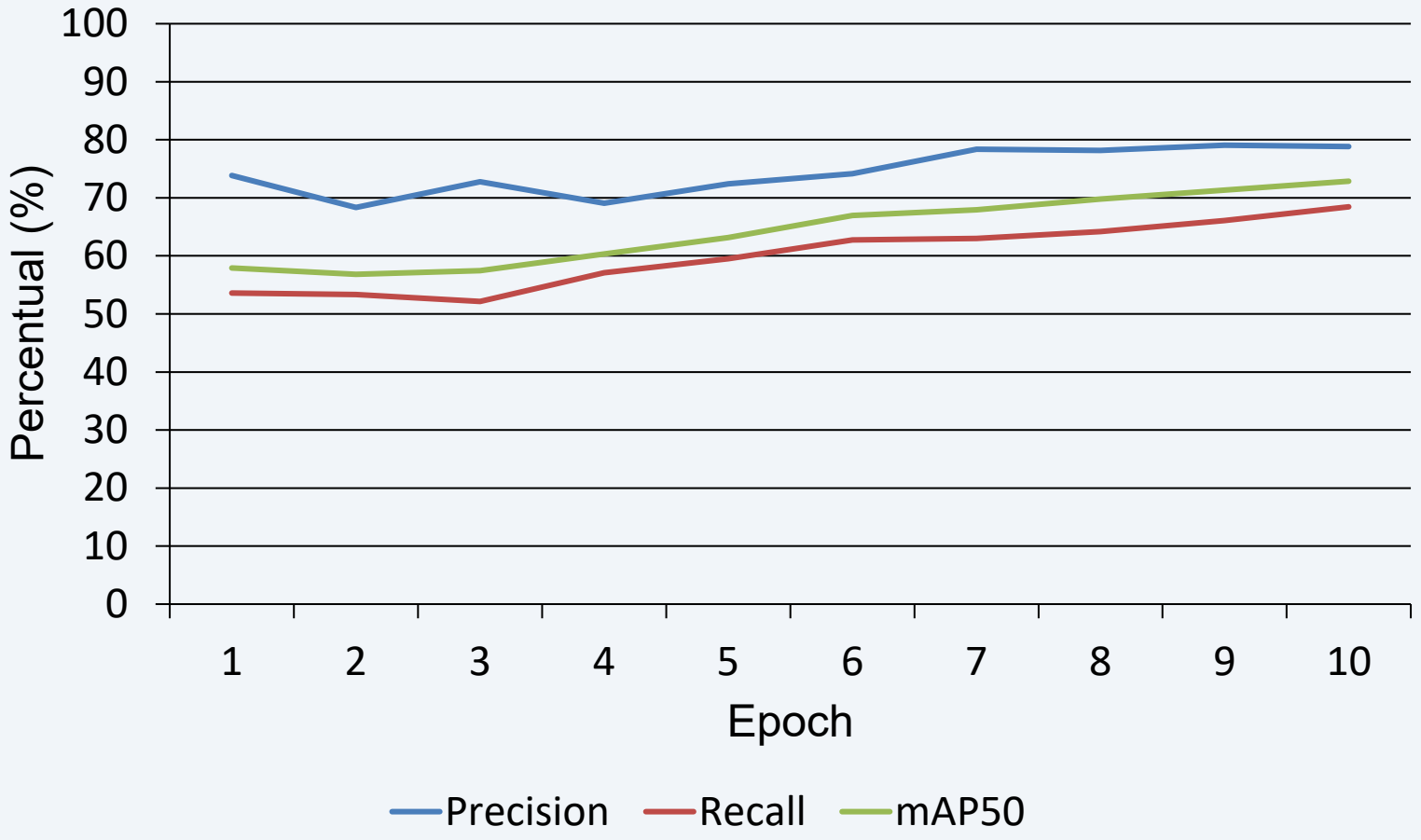
Média mais rigorosa em vários IoU.

IoU = medida de sobreposição entre a caixa prevista e a caixa correta.

Não existe uma única “acurácia” que conte toda a história. Precision alta pode coexistir com objetos não encontrados; Recall ajuda a enxergar omissões.

A evolução ao longo das 10 epochs

O CSV mostra melhora gradual nas métricas selecionadas.



Precision final

78.8%

Recall final

68.5%

mAP50 final

72.9%

Melhora geral não garante que todas as classes funcionem bem em imagens reais.

Etapa 12 — last.pt e best.pt: dois registros

O treinamento pode guardar o estado mais recente e o estado com melhor avaliação.

last.pt

O QUÊ? Checkpoint do fim da execução.

PRA QUÊ? Retomar o treinamento a partir do estado mais recente.

best.pt

O QUÊ? Checkpoint considerado melhor segundo a métrica configurada.

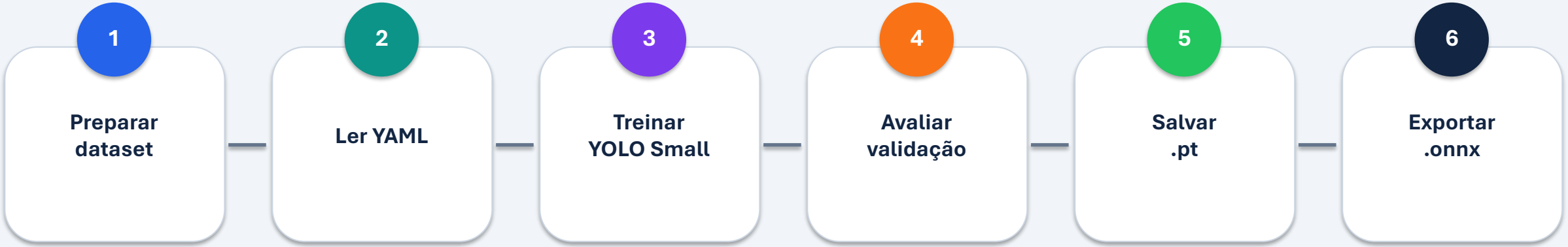
PRA QUÊ? Candidato natural para exportar e implantar.

best.pt → exportação → best.onnx

O CSV registra resultados por epoch. A confirmação de qual checkpoint foi selecionado depende da lógica de salvamento da pipeline.

Etapa 13 — A pipeline de treinamento

Pipeline = sequência organizada de tarefas que leva dos dados ao modelo exportado.



O QUÊ?

Script que automatiza o ciclo de treinamento.

PRA QUÊ?

Repetir o processo com menos passos manuais.

COMO?

Configuração + comandos + avaliação + exportação.

Automação não elimina revisão humana: confira dados, caminhos, logs e arquivos gerados.

Etapa 14 — Vibe coding com Google Antigravity

A pipeline foi criada com apoio de agente orientado por instruções do projeto.

Google Antigravity

Ambiente de desenvolvimento com agentes de IA que ajudam a planejar e editar arquivos.

Skill.md

Arquivo de instruções: contexto, padrões, restrições e critérios para orientar o agente.

Parâmetros FIT

Requisitos e condições do projeto fornecidos para orientar a solução.

Vibe coding = desenvolver descrevendo a intenção em linguagem natural e iterando com ajuda da IA.

O QUÊ?

Usar um agente para acelerar a escrita da pipeline.

COMO com responsabilidade?

Pedir, revisar, executar, ler erros e validar resultados.

O que o modelo já faz — e onde falha

O teste prático é tão importante quanto a métrica global.

Já identifica

- Pessoas
- Luvas
- Óculos

Ainda precisa melhorar

- Confunde capacetes
- Ainda não identifica coletes

O que isso ensina?

Métricas gerais podem esconder dificuldades específicas por classe.

Hipóteses a investigar (não conclusões):

- Poucos exemplos ou variedade insuficiente de capacetes/coletes.
- Anotações inconsistentes, objetos pequenos, oclusão ou iluminação difícil.
- Confusão entre classes ou diferença entre treino e cenário real.

Próximas melhorias: plano de investigação

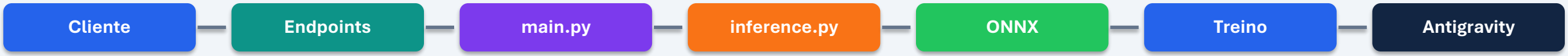
Cada melhoria deve ser testada com evidências, não apenas por tentativa.

1 • Dados	Revisar quantidade, diversidade e qualidade das anotações.
2 • Métricas por classe	Inspecionar matriz de confusão, Precision e Recall por classe.
3 • Treinamento	Comparar mais epochs e hiperparâmetros; “mais” nem sempre é melhor.
4 • Imagens reais	Criar teste independente com luz, distância e ângulos variados.
5 • Implantação	Medir latência, memória e estabilidade do YOLO Small no Raspberry Pi.

Meta: melhorar a detecção sem perder velocidade e viabilidade na borda.

Fechamento — a cadeia completa

Do resultado demonstrável à origem do modelo.



Pergunta para a turma

Se treinarmos um modelo melhor, precisamos mudar os clientes da API?

Resposta: não, se endpoints e formato das respostas continuarem compatíveis.

Mini glossário para acompanhar a aula

Termos que apareceram ao longo da apresentação.

API	Interface para programas se comunicarem.
Endpoint	Endereço/função específica dentro da API.
Inferência	Uso do modelo treinado para fazer previsão.
YOLO	Família de modelos para detectar objetos.
ONNX	Formato portátil para modelos de IA.
Epoch	Uma passagem completa pelo conjunto de treino.
Dataset	Conjunto de imagens e anotações.
YAML	Formato de texto para configuração.
Precision / Recall	Acertos entre detecções / objetos reais encontrados.
mAP / IoU	Qualidade média / sobreposição das caixas.
Checkpoint	Arquivo com parâmetros aprendidos.
GPU / T4	Processador de cálculos paralelos / modelo NVIDIA.
Vibe coding / Skill.md	Programar com IA / instruções para o agente.